

Práctica: Extendiendo el Workflow con MLflow

Objetivo

Esta práctica tiene como objetivo principal extender el workflow de la práctica anterior agregando el uso de MLflow para gestionar, comparar, y versionar modelos.

Pre-requisitos

- Haber realizado la práctica anterior:
Clase 1 - Practica de Workflow de ML con Airflow

Materiales y Herramientas

- Editor de texto/IDE (VS Code, PyCharm, etc.).
 - Terminal/Línea de comandos.
 - Docker y Docker Compose
-

1. Introducción a MLflow

Preparación:

MLflow puede ser hosteado de manera local al igual que Airflow mediante el mismo `docker-compose.yaml` con algunas modificaciones:

Agregar el servicio de MLflow. Así como el `docker-compose.yaml` instancia todos los servicios de Airflow (`airflow-apiserver`, `airflow-worker`, `airflow-dag-processor`, etc.) vamos a tener que agregar el servicio de mlflow de la siguiente manera:

Python

```
---  
x-airflow-common:
```

```
# Algunas configuraciones comunes...

services:

# Algunos servicios...

mlflow:
  image: ghcr.io/mlflow/mlflow:v3.10.1
  ports:
    - "9191:9090"
  volumes:
    - ./mlflow.db:/mlflow/mlflow.db
    - ./mlruns:/mlflow/mlruns
  command: >
    mlflow server
    --backend-store-uri sqlite:///mlflow/mlflow.db
    --default-artifact-root /mlflow/mlruns
    --host 0.0.0.0
    --port 9090
    --allowed-hosts "*"
```

Crear los volúmenes en donde MLflow guardará los experimentos realizados y la metadata.

Shell

```
mkdir mlruns
touch mlflow.db
```

“Presentar” el servicio de mlflow a los servicios de Airflow. Necesitamos decirle a airflow “dónde” está mlflow. Para esto agregar las siguientes líneas

Python

```
---
x-airflow-common:

    # Algunas configuraciones comunes...

    environment:

        # Algunas variables de entorno...

        MLFLOW_TRACKING_URI: 'http://mlflow:9090' # <---
    Agregar!

    volumes:

        # Algunos volúmenes...

        - ./mlruns:/mlflow/mlruns # <--- Agregar!

    services:

        # Algunos servicios...
```

Agregar mlflow como dependencia de Airflow para que se instale y esté disponible para usar en los DAGs. Basta con modificar la variable `_PIP_ADDITIONAL_REQUIREMENTS` agregándole `mlflow` al final. Esto se hace en el archivo `.env`

2. Agregando MLflow a nuestro Workflow

La idea de esta segunda etapa es declarar muchos experimentos, que Airflow los corra y los *logge* en MLflow de modo tal que podamos compararlos desde la UI.

Para esto les dejamos definidos algunos experimentos:

Python

```
ALL_FEATURES_GAS = ['prod_pet', 'prod_agua', 'tef',  
                    'profundidad', 'tipoextraccion']  
  
REDUCED_FEATURES_GAS = ['prod_pet', 'tef', 'profundidad']  
  
EXPERIMENTS = [  
    {'model_type': 'random_forest', 'model_params':  
    {'n_estimators': 50, 'random_state': 204}, 'target':  
    'prod_gas', 'features': ALL_FEATURES_GAS},  
    {'model_type': 'random_forest', 'model_params':  
    {'n_estimators': 100, 'random_state': 204}, 'target':  
    'prod_gas', 'features': ALL_FEATURES_GAS},  
    {'model_type': 'random_forest', 'model_params':  
    {'n_estimators': 200, 'random_state': 204}, 'target':  
    'prod_gas', 'features': ALL_FEATURES_GAS},  
    {'model_type': 'random_forest', 'model_params':  
    {'n_estimators': 100, 'random_state': 204}, 'target':  
    'prod_gas', 'features': REDUCED_FEATURES_GAS},  
]
```

Modifiquen `train_model()` para que entrene un modelo según la configuración especificada y registre los hiperparámetros, métricas y modelo en MLflow.

Para esto pueden seguir las instrucciones en el [Quickstart de MLflow](#). El ejemplo que utiliza es el siguiente:

Python

```
import mlflow  
import pandas as pd  
from sklearn import datasets  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import accuracy_score,  
precision_score, recall_score, f1_score  
  
mlflow.set_experiment("MLflow Quickstart")
```

```

# Load the Iris dataset
X, y = datasets.load_iris(return_X_y=True)

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Define the model hyperparameters
params = {
    "solver": "lbfgs",
    "max_iter": 1000,
    "random_state": 8888,
}

# Enable autologging for scikit-learn
mlflow.sklearn.autolog()

# Just train the model normally
lr = LogisticRegression(**params)
lr.fit(X_train, y_train)

```

Para logear parámetros adicionales que no son capturados por el autologger de MLflow, pueden usar la siguiente sintaxis:

```

Python
mlflow.log_param('param_name', param_value)

```

A correr los experimentos

Mientras testeen el DAG, les recomiendo dejar comentados todos los experimentos salvo el primero para ahorrar tiempo. Luego, quizás necesiten correrlos secuencialmente para evitar quedarse sin memoria.

Para esto pueden utilizar el siguiente wiring:

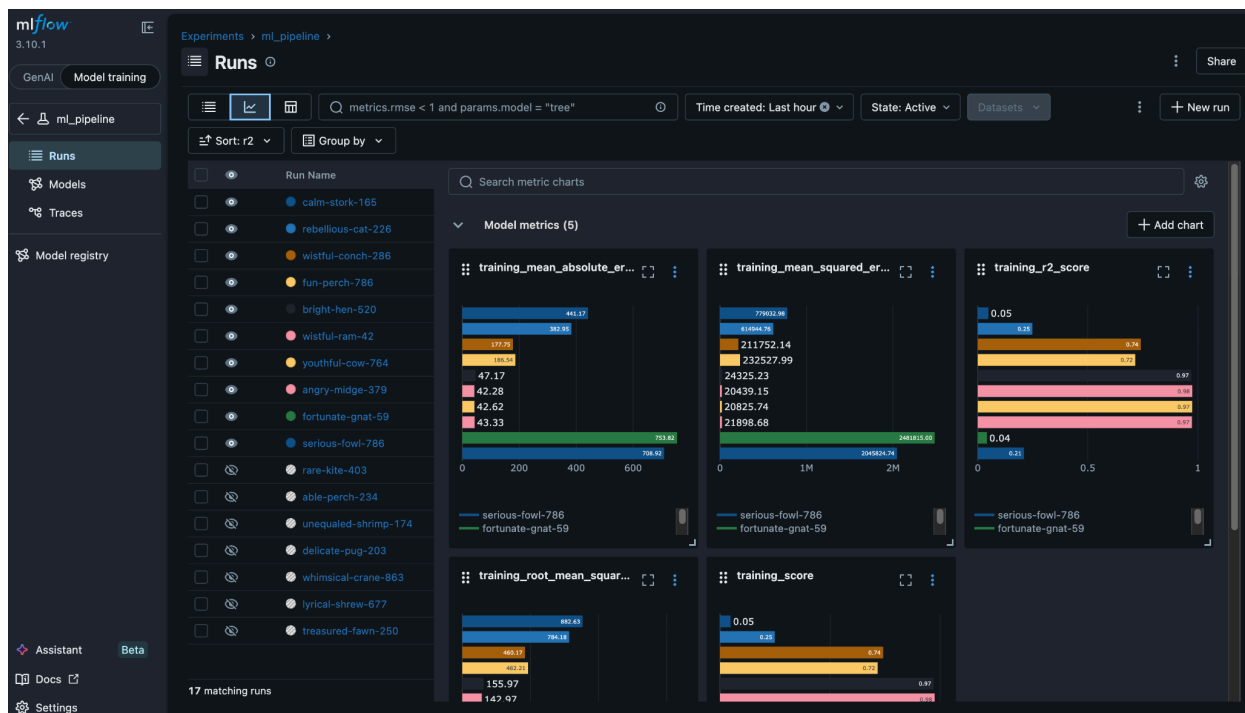
Python

```
dataset = download_dataset(DATASET_DOWNLOAD_URL,  
    '/tmp/dataset.csv')  
data = preprocess(dataset)
```

```
start >> dataset  
prev_task = data  
for exp in EXPERIMENTS:  
    result = train_model(data, config=exp)  
    prev_task >> result  
    prev_task = result
```

Visualizando los modelos

Una vez hayamos corrido suficientes experimentos, en <http://localhost:9090/#/experiments> vamos a poder ver las corridas y sus métricas.



Cada corrida tendrá:

1. Sus métricas de modelo
 - a. MAE
 - b. MSE
 - c. R^2
 - d. RMSE
 - e. Training Score (que en este caso será R^2)
2. Sus parámetros de entrenamiento
3. Su modelo asociado

Experimentos

Propongan 2 o 3 modelos y corranlos con un distintos hiper-parámetros para luego compararlos.

Extra: prueben utilizar algún framework como [Optuna](#) para encontrar los hiper-parámetros óptimos.